take the reference of https://dribbble.com/shots/26332697-Real...

Okay, this is an excellent specific reference! The Dribbble shot provides a clear visual direction for the Rentro platform's user interface.

Taking inspiration from the Dribbble shot "Real Estate Website Design for Property Rentals and Apartments" (https://dribbble.com/shots/26332697-Real-Estate-Website-Design-for-Property-Rentals-and-Apartments), here's an updated Design Document for Rentro, with a strong emphasis on the UI/UX and Frontend Design sections to reflect this visual style.

Design Document: Rentro - PG Accommodation Platform

Document Version: 1.1 (Visual Update)
Date: March 28, 2026
Lead Architect/Engineer: [Your Name/Team Lead]
Reviewers: [Relevant Stakeholders, e.g., CTO, Senior Devs, UI/UX Lead]

---

1. Introduction

This document outlines the technical design for the Rentro web application, a platform connecting PG seekers and owners. It translates the functional and non-functional requirements from the Product Requirements Document (PRD) into a concrete architectural and implementation plan. This design focuses on building a scalable, secure, performant, and maintainable system, prioritizing the features defined in the MVP, **while adhering to the modern, clean, and intuitive visual design cues from the referenced Dribbble shot.**

2. Goals & Principles

2.1. Technical Goals

- **Scalability:** Design for high user concurrency and rapid data growth.
- **Performance:** Achieve fast load times and responsive interactions.

🚀 Powered by Yupp

- **Security:** Implement robust security measures to protect user data and prevent unauthorized access.
- **Maintainability:** Use clean code, modular architecture, and comprehensive documentation.
- **Reliability:** Ensure high availability and data integrity.
- **Extensibility:** Allow for easy integration of new features and third-party services.

2.2. Design Principles

- **Cloud-Native First:** Leverage cloud services for infrastructure management, scalability, and resilience.
- **API-Centric:** All components communicate via well-defined APIs.
- **Separation of Concerns:** Clearly delineate responsibilities between different layers and components.
- **Automation:** Automate testing, deployment, and infrastructure provisioning (IaC).
- **Observability:** Implement robust monitoring, logging, and tracing.
- **User-Centric Visuals:** Prioritize a clean, intuitive, and aesthetically pleasing user interface, heavily influenced by the provided Dribbble reference, to enhance user experience and trust.

2.3. Visual Reference & Inspiration

The primary visual inspiration for Rentro's frontend design is the Dribbble shot:
"Real Estate Website Design for Property Rentals and Apartments"
URL: `https://dribbble.com/shots/26332697-Real-Estate-Website-Design-for-Property-Rentals-and-Apartments`

This design emphasizes:

- **Clean, modern aesthetics** with ample whitespace.
- **Intuitive navigation** and prominent search functionality.
- **High-quality imagery** for property listings.
- **Professional and inviting** color palette and typography.
- **Clear hierarchy** and information presentation on listing cards and detail pages.

3. Architecture Overview

Rentro will adopt a **microservices-oriented architecture (SOA)**, allowing independent development, deployment, and scaling of core functionalities. For the MVP, some services might be initially co-located or combined, evolving into distinct microservices as the platform scales.

🚀 Powered by Yupp

A **single-page application (SPA)** will be used for the frontend, communicating with backend services via RESTful APIs.

3.1. High-Level Architecture Diagram

```mermaid
graph TD
    User(User - Seeker/Owner) --> |Web Browser| CDN(CDN - Static Assets)
    User --> |Web Browser| LB(Load Balancer)

    LB --> FE(Frontend Service - Next.js)
    FE --> |API Calls| API_GW(API Gateway)

    API_GW --> AuthS(Authentication Service)
    API_GW --> UserS(User Service)
    API_GW --> ListingS(Listing Service)
    API_GW --> InquiryS(Inquiry Service)
    API_GW --> NotifS(Notification Service)
    API_GW --> AdminS(Admin Service)

    subgraph Data Stores
        AuthS & UserS & ListingS & InquiryS & NotifS & AdminS --> PostgreSQL(PostgreSQL Cluster)
        PostgreSQL --- Redis(Redis Cache)
        PostgreSQL --- Elastic(Elasticsearch for Search)
        PostgreSQL --- S3(S3/Cloud Storage - Media)
    end

    NotifS --> Email_Provider(External Email Provider)
    ListingS --> Map_API(Google Maps API)

    Ops(Operations & Monitoring) -- Alerts & Logs --> Prometheus(Prometheus)
    Ops -- Logs --> Grafana(Grafana)
    Ops -- Tracing --> Jaeger(Jaeger)
    Ops -- CI/CD --> GitHub_Actions(GitHub Actions)

```

3.2. Architectural Layers

1. **Client Layer:** Web browser interacting with the application.
2. **CDN Layer:** Serves static assets (images, CSS, JS) to improve load times and reduce origin server load.
3. **Edge Layer:** Load Balancer and API Gateway for request routing, security, and rate limiting.

🚀 Powered by Yupp

4. **Application Layer (Microservices):** Backend services implementing business logic.
5. **Data Layer:** Databases, caching, search engine, and object storage.
6. **External Services Layer:** Third-party APIs (e.g., email, maps, payment gateways).
7. **Operations Layer:** Monitoring, logging, CI/CD, infrastructure management.

4. Component Breakdown

4.1. Frontend Design & UI/UX (Influenced by Dribbble Shot)

- **Technology:** Next.js (React Framework) with TypeScript. Tailwind CSS for utility-first styling.
- **Purpose:** Provide a visually appealing, responsive, interactive, and SEO-friendly user interface for seekers and owners, consistent with the Dribbble reference.
- **Key Responsibilities:**
  - Client-side rendering (CSR) and Server-Side Rendering (SSR) for initial page loads (SEO benefits).
  - State management (e.g., React Context API, Zustand, or Redux Toolkit).
  - Form validation and user input handling.
  - API integration and data presentation.
  - Routing and navigation.
- **Visual Design Principles (Derived from Dribbble Shot):**
  - **Color Palette:**
    - **Primary Accent:** A professional, inviting blue (e.g., `#007bff` or similar vibrant shade). Used for primary buttons, links, active states, and key iconography.
    - **Backgrounds:** Predominantly clean white (`#FFFFFF`) or very light off-white/gray (`#F8F9FA`) for content areas to ensure readability and spaciousness.
    - **Text:** Dark gray (`#333333` or similar) for main body text, lighter grays for secondary text or descriptions.
    - **Accents:** Subtle greens/teals for "available" states or positive feedback; soft reds for warnings/errors.
  - **Typography:**
    - **Fonts:** Modern sans-serif typeface (e.g., Poppins, Inter, Montserrat, or a similar Google Font).
    - **Weights:** Strategic use of different font weights (light, regular, semi-bold, bold) to establish hierarchy and emphasize important information.

🚀 Powered by Yupp

    - **Readability:** Generous line heights and letter spacing to improve readability.
- **Layout & Spacing:**
    - **Whitespace:** Ample whitespace around elements to create a clean, uncluttered look.
    - **Grids:** Utilize a responsive grid system (e.g., Flexbox, CSS Grid) to ensure consistent layout across devices.
    - **Sections:** Clearly defined sections with subtle dividers or background variations.
- **Imagery:**
    - **High-Quality Photos:** Emphasize high-resolution, bright, and inviting property images on listing cards and detail pages. Image aspect ratios should be consistent for a uniform grid.
    - **Illustrations/Icons:** Modern, clean line-art icons for features, amenities, and navigation. SVG format preferred for scalability.
- **Component Styling:**
    - **Buttons:** Primary buttons with the accent blue, rounded corners, clear hover states. Secondary buttons with outlined styles.
    - **Input Fields:** Clean, minimal input fields with subtle borders and clear focus states. Placeholder text in a lighter shade.
    - **Listing Cards:**
        - Card-based layout (similar to Dribbble reference).
        - Prominent image at the top.
        - Clear overlay for price or key information.
        - Well-structured text area for title, location, and key features (e.g., "beds," "baths" - adapted to "sharing capacity," "amenities" for PG).
        - Subtle shadows/lifts on hover for interactivity.
    - **Navigation Bar:** Fixed header with logo (Rentro branding), clear navigation links (e.g., "Find PG", "List Your PG", "Dashboard"), and a prominent "Login/Register" or "Add Listing" call-to-action button.
    - **Search Bar (Hero Section):** A large, visually dominant search bar in the hero section, similar to the Dribbble reference, allowing users to input location, and potentially quick filters like "Gender Preference" or "Room Type."
- **Responsiveness:**

🚀 Powered by Yupp

    - Fully responsive design, adapting gracefully from large desktop screens to mobile phones.
    - Touch-friendly interactions for mobile users.
- **Design Considerations:**
    - **Atomic Design Principles:** Build reusable UI components (atoms, molecules, organisms, templates, pages) for consistency and speed.
    - **Performance:** Lazy loading of components, image optimization (WebP, AVIF formats), code splitting, and efficient data fetching.
    - **Accessibility:** Adherence to WCAG guidelines for inclusive design (keyboard navigation, ARIA attributes, sufficient color contrast).
    - **Error Handling:** Graceful error display and recovery mechanisms, consistent with the visual style.
    - **Micro-interactions:** Subtle animations and transitions to enhance user feedback and engagement.
- **Interactions:** Communicates with the API Gateway via HTTP/HTTPS requests.

4.2. Backend Services (Node.js with NestJS)

Each service will be a separate NestJS application.

4.2.1. Authentication Service (`AuthService`)

- **Purpose:** Handle user authentication and authorization.
- **Key Responsibilities:**
    - User registration (email/password, social login - optional for MVP).
    - User login, session management (JWT tokens).
    - Password hashing (Bcrypt).
    - Password reset functionality.
    - Role-based authorization checks.
- **Database:** Dedicated `AuthDB` (PostgreSQL) for user credentials and session data.
- **Design Considerations:** Stateless JWT for API authentication, refresh tokens for long-lived sessions, secure token storage (HTTP-only cookies).

4.2.2. User Service (`UserService`)

- **Purpose:** Manage user profiles for both seekers and owners.

🚀 Powered by Yupp

- **Key Responsibilities:**
  - Create, retrieve, update, delete user profiles.
  - Store user preferences and settings.
  - Handle profile picture uploads (delegates to S3).
- **Database:** `UserDB` (PostgreSQL) for user profile data.
- **Interactions:** Called by `AuthService` during registration; accessed by `ListingService` (to get owner details) and `InquiryService` (to get seeker details).

4.2.3. Listing Service (`ListingService`)

- **Purpose:** Manage PG listings.
- **Key Responsibilities:**
  - CRUD operations for PG listings.
  - Handle media uploads (images/videos) to S3.
  - Integrate with Google Maps API for location data.
  - Update Elasticsearch index on listing changes.
  - Availability management for rooms/beds.
- **Database:** `ListingDB` (PostgreSQL) for all PG listing details.
- **Interactions:** Relies on `UserService` for owner details, interacts with S3 for media, Google Maps API for geo-coding, Elasticsearch for indexing.

4.2.4. Inquiry Service (`InquiryService`)

- **Purpose:** Manage communication inquiries between seekers and owners.
- **Key Responsibilities:**
  - Create and retrieve inquiries.
  - Track inquiry status (sent, read, responded).
- **Database:** `InquiryDB` (PostgreSQL) for inquiry messages and metadata.
- **Interactions:** Called by `UserService` (seeker) and `ListingService` (owner) for display. Triggers `NotificationService`.

4.2.5. Notification Service (`NotificationService`)

- **Purpose:** Send email notifications to users.
- **Key Responsibilities:**

🚀 Powered by Yupp

    - Queue and send email notifications (e.g., inquiry received, inquiry responded, password reset).
    - Manage notification templates.
- **Database:** `NotificationDB` (PostgreSQL) for notification logs and template management.
- **Interactions:** Consumes events/messages from other services (e.g., `InquiryService`) via a message queue (e.g., Redis Pub/Sub or Kafka). Integrates with a third-party email provider (e.g., SendGrid, AWS SES).

4.2.6. Admin Service (`AdminService`)

- **Purpose:** Provide administrative functionalities.
- **Key Responsibilities:**
    - User management (view, deactivate, delete users).
    - Listing moderation (approve, reject, edit listings).
    - Content management for FAQ, T&C.
    - Basic analytics aggregation (from other services' databases or logs).
- **Database:** Can use shared read-only access to other service databases or have its own `AdminDB` for admin-specific data (e.g., moderation logs).
- **Interactions:** Interacts with `UserService` and `ListingService` via their APIs.

4.3. API Gateway

- **Technology:** AWS API Gateway / Nginx / Express.js Proxy.
- **Purpose:** Act as the single entry point for all client requests, routing them to the appropriate backend service.
- **Key Responsibilities:**
    - Request routing based on path.
    - Authentication (validate JWTs, pass user context to services).
    - Rate limiting.
    - Load balancing requests to service instances.
    - Security (WAF integration).

4.4. Data Layer

4.4.1. PostgreSQL (Relational Database)

- **Purpose:** Primary data store for all structured application data.

🚀 Powered by Yupp

- **Deployment:** Managed PostgreSQL service (AWS RDS, GCP Cloud SQL) for high availability, backups, and scaling.
- **Design Considerations:**
  - **Schema Per Service:** Each microservice owns its schema and tables. No direct cross-service database access.
  - **Indexing:** Appropriate indexing for frequently queried columns (e.g., `user_id`, `pg_id`, `location`).
  - **Transactions:** Ensure data consistency where necessary.
  - **Migrations:** Use tools like `TypeORM Migrations` or `Knex.js` for schema evolution.

4.4.2. Redis (In-Memory Data Store / Cache)

- **Purpose:** Caching frequently accessed data, session management, real-time data, message queue (Pub/Sub).
- **Deployment:** Managed Redis service (AWS ElastiCache, GCP Memorystore).
- **Design Considerations:**
  - **Cache Invalidation:** Implement strategies for stale data.
  - **Session Management:** Store JWT refresh tokens and blacklist revoked tokens.
  - **Pub/Sub:** For internal service communication (e.g., triggering notifications).

4.4.3. Elasticsearch (Search Engine)

- **Purpose:** Provide fast and flexible full-text search capabilities for PG listings.
- **Deployment:** Managed Elasticsearch service (AWS OpenSearch, Elastic Cloud).
- **Design Considerations:**
  - **Indexing:** `ListingService` will push relevant data to Elasticsearch upon creation or update of PG listings.
  - **Mapping:** Define appropriate mappings for text fields, numbers, and geo-location data.
  - **Query DSL:** Leverage Elasticsearch's powerful query language for complex search and filtering.

4.4.4. S3 / Cloud Storage (Object Storage)

- **Purpose:** Store static media files (PG images, videos, user profile pictures).
- **Deployment:** AWS S3, GCP Cloud Storage.

🚀 Powered by Yupp

- **Design Considerations:**
  - **Pre-signed URLs:** Generate temporary, secure URLs for direct uploads/downloads to S3 to offload backend services.
  - **Access Control:** Proper bucket policies and IAM roles.
  - **CDN Integration:** Distribute media via CDN for faster delivery.

4.5. Cloud Infrastructure (AWS - Example)

- **Compute:** AWS EC2 instances (for NestJS services, Next.js build) or AWS Fargate (serverless containers).
- **Load Balancing:** AWS Application Load Balancer (ALB).
- **API Gateway:** AWS API Gateway.
- **Databases:** AWS RDS for PostgreSQL, AWS ElastiCache for Redis, AWS OpenSearch for Elasticsearch.
- **Storage:** AWS S3 for media files.
- **CDN:** AWS CloudFront.
- **DNS:** AWS Route 53.
- **Container Orchestration:** AWS EKS (Kubernetes) or AWS ECS (for managing Docker containers). For MVP, ECS with Fargate might be simpler.
- **Monitoring & Logging:** AWS CloudWatch, Prometheus, Grafana.

5. Data Model Design (High-Level)

Conceptual relationships between key entities:

- **User:**
  - `id` (PK)
  - `email` (Unique)
  - `password_hash`
  - `role` (ENUM: 'seeker', 'owner', 'admin')
  - `name`
  - `phone_number`
  - `profile_picture_url`
  - `created_at`, `updated_at`
- **PG_Listing:**

🚀 Powered by Yupp

    - `id` (PK)
    - `owner_id` (FK to User.id)
    - `title`
    - `description`
    - `address` (Street, Locality, City, State, Pincode)
    - `latitude`, `longitude`
    - `rent_per_month`
    - `deposit_amount`
    - `notice_period_days`
    - `gender_preference` (ENUM: 'male', 'female', 'co-ed')
    - `room_types_available` (Array of ENUM: 'single', 'double', 'triple')
    - `amenities` (JSONB or separate `PG_Amenity` table for detailed attributes)
    - `rules`
    - `status` (ENUM: 'active', 'inactive', 'pending_approval')
    - `media_urls` (Array of URLs to S3)
    - `created_at`, `updated_at`
- **Inquiry:**
    - `id` (PK)
    - `seeker_id` (FK to User.id)
    - `pg_listing_id` (FK to PG_Listing.id)
    - `message`
    - `status` (ENUM: 'pending', 'responded', 'closed')
    - `created_at`, `updated_at`
- **Shortlist:**
    - `seeker_id` (FK to User.id) (Composite PK with pg_listing_id)
    - `pg_listing_id` (FK to PG_Listing.id) (Composite PK with seeker_id)
    - `created_at`

6. API Design (High-Level)

All APIs will be RESTful, using JSON for request/response bodies and standard HTTP methods.

🚀 Powered by Yupp

6.1. Authentication

- `POST /auth/register`: Register a new user.
- `POST /auth/login`: Log in, returns JWT.
- `POST /auth/refresh-token`: Refresh JWT.
- `POST /auth/forgot-password`: Initiate password reset.
- `POST /auth/reset-password`: Reset password.

6.2. Users

- `GET /users/me`: Get current user's profile. (Authenticated)
- `PUT /users/me`: Update current user's profile. (Authenticated)
- `GET /users/{id}`: Get user profile by ID (e.g., owner profile for listing).

6.3. Listings

- `POST /listings`: Create a new PG listing. (Owner, Authenticated)
- `GET /listings/{id}`: Get a single PG listing by ID.
- `PUT /listings/{id}`: Update a PG listing. (Owner, Authenticated)
- `DELETE /listings/{id}`: Delete a PG listing. (Owner, Authenticated)
- `GET /listings/my`: Get all listings for the current owner. (Owner, Authenticated)
- `GET /listings/search`: Search and filter PG listings.

6.4. Inquiries

- `POST /inquiries`: Send an inquiry for a PG. (Seeker, Authenticated)
- `GET /inquiries/my`: Get all inquiries sent by seeker. (Seeker, Authenticated)
- `GET /inquiries/received`: Get all inquiries received by owner. (Owner, Authenticated)
- `PUT /inquiries/{id}/respond`: Respond to an inquiry. (Owner, Authenticated)

6.5. Shortlist

- `POST /shortlist/{listingId}`: Add a PG to shortlist. (Seeker, Authenticated)
- `DELETE /shortlist/{listingId}`: Remove a PG from shortlist. (Seeker, Authenticated)
- `GET /shortlist`: Get all shortlisted PGs. (Seeker, Authenticated)

7. Security Design

- **HTTPS Everywhere:** All traffic encrypted using SSL/TLS.

🚀 Powered by Yupp

- **Authentication:** JWT-based authentication with refresh tokens. Token validation at API Gateway.
- **Authorization:** Role-based access control (RBAC) enforced by services and API Gateway.
- **Password Hashing:** Use strong, salted hashing algorithm (Bcrypt).
- **Input Validation:** Strict server-side validation for all user inputs to prevent injection attacks (SQLi, XSS).
- **CORS:** Properly configured Cross-Origin Resource Sharing.
- **Rate Limiting:** Protect against brute-force attacks and abuse at the API Gateway level.
- **Secrets Management:** Environment variables or a dedicated secrets manager (AWS Secrets Manager, HashiCorp Vault) for sensitive credentials.
- **Dependency Scanning:** Regularly scan third-party libraries for known vulnerabilities.
- **Regular Audits:** Scheduled security audits and penetration testing.
- **Data Masking:** Mask sensitive user contact details from general view until an inquiry is made.

8. Scalability & Performance Design

- **Microservices:** Independent scaling of services based on demand.
- **Stateless Services:** Design services to be stateless to enable easy horizontal scaling.
- **Load Balancing:** Distribute incoming traffic across multiple instances of services.
- **Caching:**
  - **CDN:** For static assets (images, JS, CSS).
  - **Redis:** For frequently accessed data (e.g., popular listings), user sessions.
- **Database Scaling:**
  - **PostgreSQL:** Managed service with read replicas for read-heavy workloads. Sharding if single instance performance becomes a bottleneck (long-term).
  - **Elasticsearch:** Distributed search engine for high-performance search.
- **Asynchronous Processing:** Use message queues for non-critical, time-consuming tasks (e.g., email notifications, image processing, analytics data ingestion) to avoid blocking user requests.
- **Optimized Queries:** Efficient database schema, proper indexing, and optimized queries.
- **Image Optimization:** Resize and compress images upon upload for web delivery.

9. Deployment & Operations

🚀 Powered by Yupp

- **Containerization:** All microservices and frontend packaged as Docker containers.
- **Orchestration:** Kubernetes (AWS EKS) or ECS Fargate for automated deployment, scaling, and management of containers.
- **CI/CD Pipeline:**
  - **Version Control:** GitHub.
  - **CI (Continuous Integration):** GitHub Actions to run tests, linting, and build Docker images upon code pushes.
  - **CD (Continuous Deployment):** GitHub Actions to deploy new versions of services to staging and production environments.
  - **Infrastructure as Code (IaC):** Terraform or AWS CloudFormation for provisioning and managing cloud resources.
- **Monitoring & Logging:**
  - **Centralized Logging:** Aggregate logs from all services using tools like AWS CloudWatch Logs, ELK Stack (Elasticsearch, Logstash, Kibana), or Grafana Loki.
  - **Metrics & Alerts:** Collect application and infrastructure metrics (CPU, memory, request latency) using Prometheus and visualize with Grafana. Set up alerts for critical issues.

🚀 Powered by Yupp